PROJECT DOCUMENTATION

BLOGNEST – AI POWERED BLOG CONTENT MANAGEMENT SYSTEM

INTRODUCTION :

TITLE : AI POWERED BLOG CONTENT MANAGEMENT

TEAM LEADER : VINOTHINI.V

TEAM MEMBERS :

Vinothini V - vinothinivinothini98558@gmail.com

Monisha V - vmonisha720@gmail.com

Prithika M.V - valli24584@gmail.com

Hema U – khemalatha230@gmail.com

2. **INTRODUCTION**

BlogNest is a backend-based blogging platform that provides APIs for creating, managing, and accessing blog content. The application follows a modular and scalable architecture.

The backend acts as a centralized API layer between users and the database. It manages authentication, authorization, blog operations, comments, and AI-based content services.

The system uses JWT for authentication and bcrypt.js for secure password hashing. Different users are provided with different permissions based on their roles.

3. **PROBLEM STATEMENT**

Traditional blog management can become difficult when handling multiple posts, drafts, categories, tags, comments, and user permissions.

The major problems include:

- Difficulty in managing multiple drafts, categories, and tags.
- Limited visibility into post performance.
- Time-consuming content approval and comment moderation.
- Difficulty in managing media information.
- Lack of different permission levels for administrators, editors, authors, and readers.

4. OBJECTIVES

The main objectives of BlogNest are:

- To provide a centralized blog management system.
- To simplify blog creation and content management.
- To provide AI-assisted blog content generation.
- To provide AI-based content summarization.
- To implement secure user authentication.
- To provide role-based access control.
- To manage comments and content efficiently.
- To maintain structured and secure database records.

5. EXISTING SYSTEM

In a basic blogging system, users may have limited facilities for managing content, comments, categories, and permissions. Content creation may also require more manual effort.

There may be limited support for AI-assisted content generation and summarization.

6. **PROPOSED SYSTEM**

BlogNest provides a centralized and secure backend system for blog management. It supports different user roles and provides controlled access to different operations.

The system includes blog CRUD operations, comment management, categorization, authentication, authorization, AI-powered blog generation, and content summarization.

7. **SOFTWARE REQUIREMENTS**

- Operating System: Windows 10/11, macOS, or Linux
- Node.js: Version 16 or above
- npm: Version 8 or above
- Express.js
- MongoDB
- Mongoose
- Postman
- Visual Studio Code or similar code editor

8. **HARDWARE REQUIREMENTS**

- Processor: Intel Core i5 8th Generation or above / AMD Ryzen 5 or equivalent

- RAM: Minimum 8 GB

- Recommended RAM: 16 GB

- Storage: Minimum 1 GB available workspace

9. **SYSTEM ARCHITECTURE**

BlogNest follows a layered server architecture. Incoming requests are processed through the Express server, authentication and role-based middleware, business logic services, database interface, and AI service integration.

The major components are:

- API Interface

- Express Server Gateway

- Authentication and Role Middleware

- Business Logic Services

- Database Interface Layer

- AI Flow Service

- Blog Generator with AI

10. MODULES

10.1 User Management Module

This module manages user registration, login, authentication, and user roles.

10.2 Blog Management Module

This module provides operations for creating, reading, updating, and managing blog posts.

10.3 Comment Management Module

Users can add comments to blogs. Authorized users can manage and moderate comments.

10.4 Role Management Module

The system supports four roles:

- Admin
- Editor
- Author
- Reader

Each role has different permissions.

10.5 AI Blog Generation Module

The AI module generates structured blog content from high-level topics using an external AI service.

10.6 Blog Summarization Module

The summarization service processes blog content and generates concise summaries.

11. **DATABASE DESIGN**

BlogNest uses MongoDB as its primary database and Mongoose for schema definition and database interaction.

User Collection

- _id
- name
- email
- password
- role

Post Collection

- _id
- userID
- name
- photo
- message
- likes
- timestamps

A user can create multiple posts, and users can also create multiple comments. A post can contain multiple comments.

12. **MVC ARCHITECTURE**

BlogNest follows the MVC architecture pattern.

Model

The Model layer defines the database schemas using Mongoose and manages the structure of data stored in MongoDB.

Controller

The Controller handles requests, validates inputs, communicates with models, and returns JSON responses.

View

Since BlogNest is a headless backend service, it does not use traditional UI views. The API routing layer acts as the interface between requests and controllers.

13. **SECURITY FEATURES**

BlogNest provides several security mechanisms:

- JWT-based authentication
- Role-based authorization
- Password hashing using bcrypt.js

- Input validation
- Input sanitization
- Rate limiting
- Structured logging

These features help protect the application from unauthorized access and common web vulnerabilities.

14. **API TESTING**

Postman or Thunder Client can be used to test the APIs.

The documented API operations include:

- User Registration
- User Login
- Create Blogs
- Fetch Blogs
- Generate AI Blogs
- Summarize Blogs

15. **ADVANTAGES**

- Easy blog content management.
- AI-assisted content generation.
- AI-based summarization.

- Secure authentication.
- Role-based access control.
- Structured database management.
- Comment moderation.
- Scalable backend architecture.

16. **FUTURE ENHANCEMENT**

The system can be further enhanced by adding more AI-based content features, advanced analytics, additional content management capabilities, and improved user-facing interfaces.

17. **CONCLUSION**

BlogNest provides a secure and scalable backend solution for modern blog content management. It combines traditional blogging features with Generative AI to simplify content creation and management.

The integration of Node.js, Express.js, MongoDB, Mongoose, JWT authentication, role-based authorization, and Google Gemini AI makes the system suitable for efficient and intelligent content management.